



Politechnika Łódzka
Instytut Automatyki

Laboratorium Robotów Mobilnych

Instrukcja 5

Łódź, 2019

Wstęp

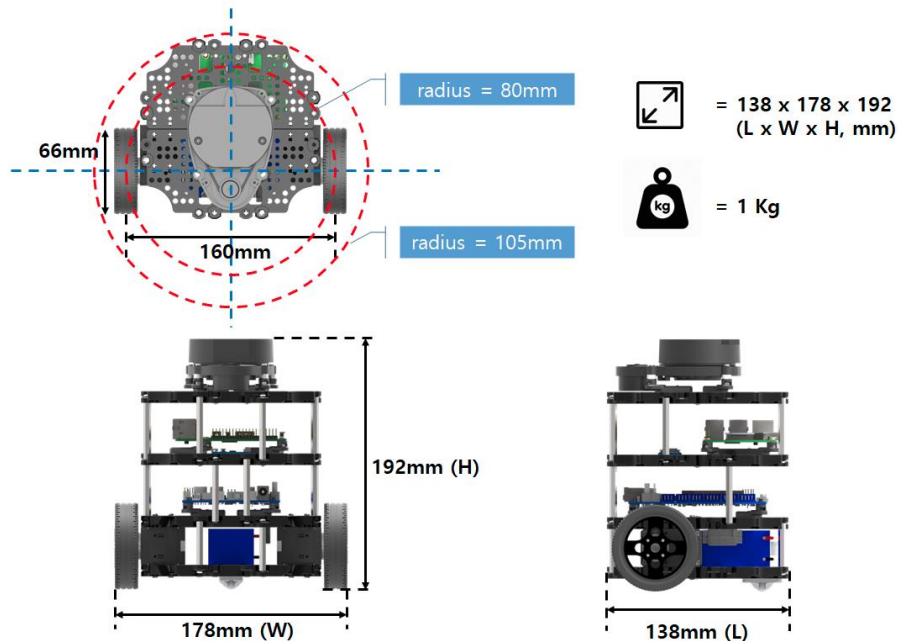
Celem niniejszych zajęć jest zaznajomienie się z kinematyką robota o napędzie różnicowym.

Dane techniczne robota

Najważniejsze dane techniczne dostępne na stronie producenta zostały przedstawione poniżej:

(<http://emanual.robotis.com/docs/en/platform/turtlebot3/specifications/>)

TurtleBot3 Burger



- maksymalna prędkość liniowa dla Burgera wynosi: 0.22 m/s
- maksymalna prędkość obrotowa dla Burgera wynosi: 2.84 rad/s

Symulacja

Zadanie 1

Bazując na plikach z poprzednich zajęć oraz wykorzystując wiedzę zdobytą na wykładach należy napisać publishera, który będzie publikował wiadomość typu Twist na topicu /cmd_vel z wartościami prędkości liniowej i kątowej robota wiedząc, że:

- robot ma poruszać się po okręgu
- promień obrotu ma wynosić 15, 20, 25, 30, 35 cm (każda grupa jedna wartość)

W celu sprawdzenia poprawności napisanego skryptu należy wykorzystać symulację. Po odpaleniu

```
roscore
```

w jednym terminalu, w kolejnym wpisujemy poniższą komendę uruchamiając tym samym robota w świecie bez przeszkód:

```
roslaunch turtlebot3_gazebo turtlebot3_empty_world.launch
```

W momencie, gdy zauważalnym w symulacji będzie ruch robota po okręgu, należy zweryfikować poprawność jego poruszania się po zadanym promieniu. Do tego celu powinno się wykorzystać dwa dostarczone przez prowadzącego skrypty (path_saver i plot_path). Po uruchomieniu pliku path_saver

zostanie zgrana do pliku circle.csv ścieżka, po której porusza się robot. Następnie przy użyciu pliku plot_path jesteśmy w stanie wyrysować trajektorię robota i porównać ją do zadanego okręgu.

Zadanie 2

Należy rozszerzyć kod z Zadania 1 tak, aby przy użyciu funkcji rospy.loginf() lub print wyświetlać prędkości koła lewego i prawego z jakimi obecnie porusza się robot (uzyskanie tych prędkości wymaga dodatkowych obliczeń).

Uruchomienie rzeczywistego robota

UWAGA!

Punkt nie dotyczy maszyny wirtualnej.

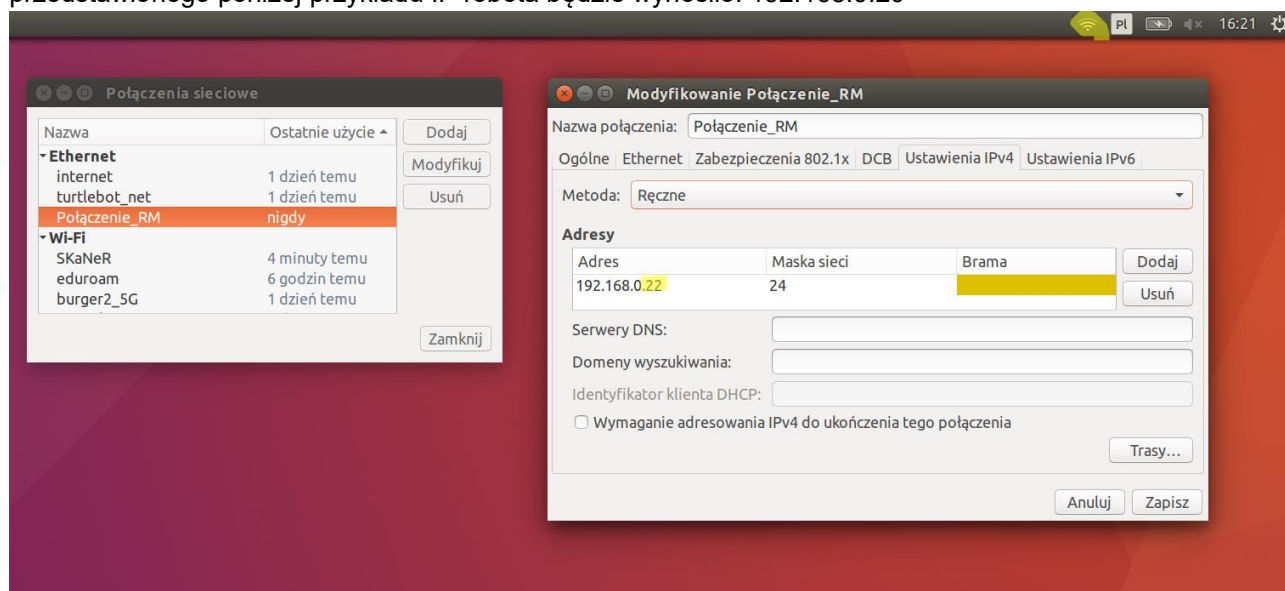
Należy zamknąć dotychczasowe uruchomione okna wraz z roscorem. W nowym terminalu należy przejść i dokonać na końcu edycji pliku .bashrc. W tym celu w terminalu wpisujemy:

```
sudo gedit ~/.bashrc
```

Należy zakomentować (#) dwie linie zawierające export ROS_MASTER_URI oraz ROS_HOSTNAME zawierające w wartościach nazwy localhost. Należy odkomentować dwie linie zawierające wpisane konkretne wartości adresów IP. Należy zapisać zmiany i zamknąć. W tym samym terminalu wpisujemy:

```
source ~/.bashrc
```

Uruchamiamy nowy terminal z **roscorem**. Uruchamiamy robota, a następnie w kolejnym terminalu sprawdzamy czy udało się nawiązać połączenie z robotem z wykorzystaniem polecenia ping. Najpierw należy ustalić adres IP robota, który zależy od IP stanowiska. Do sprawdzenia adresu IP stanowiska wybieramy zaznaczoną na górnym pasku opcję od połączenia -> wybieramy modyfikuj połączenia -> następnie Połączenie_RM -> modyfikuj. Przechodzimy do zaznaczonej zakładki i sprawdzamy IP. Interesują nas ostatnie dwie cyfry. IP robota różni się od IP komputera ostatnią cyfrą tzn. dla przedstawionego poniżej przykładu IP robota będzie wynosiło: 192.168.0.20



Do weryfikacji połączenia służy polecenie ping. Przykład przedstawiono poniżej (należy podać IP swojego robota):

```
ping 192.168.0.1
```

Jeśli połączenie zostało nawiązane pomyślnie i odbierane są dane od urządzenia należy zalogować się na robota przez ssh. Przykład poniżej (należy zmienić i podać IP robota):

```
ssh pi@192.168.0.1
```

Następnie należy wpisać hasło:

```
turtlebot
```

Po zalogowaniu się na robota może być konieczna synchronizacja czasu robota z serwerem czasu znajdującym się na komputerze. Polecenie do sprawdzenia aktualnej daty to:

```
date
```

Jeśli widoczna jest różnica pomiędzy czasem komputera, a czasem na naszym robocie należy tą różnicę zniwelować. W tym celu należy wykorzystać polecenie do synchronizacji i zamiast przykładowego IP wpisać IP komputera:

```
sudo ntpdate -u 192.168.0.1
```

Po zweryfikowaniu i zsynchronizowaniu czasów w terminalu z połączeniem robota uruchamiamy noda odpowiedzialnego za uruchomienie turtlebota i jego funkcji:

```
roslaunch turtlebot3_bringup turtlebot3_robot.launch
```

Należy uruchomić na komputerze napisany skrypt z prędkościami sterującymi robotem i zaobserwować ruch robota na podłodze. Powinien cały czas zataczać jednakowe okręgi. Można spróbować zaobserwować co się dzieje z robotem podczas dłuższego działania. Czy po kilku okrążeniach nie następuje przesuwanie się okręgów w którąś stronę?

Omijanie przeszkód

Należy ponownie powrócić do środowiska symulacyjnego i przygotować algorytmy odpowiedzialne za jazdę i omijanie przeszkód. W tym celu należy przywrócić plik `.bashrc` do poprzedniego stanu (to co zostało zakomentowane należy odkomentować i odwrotnie).

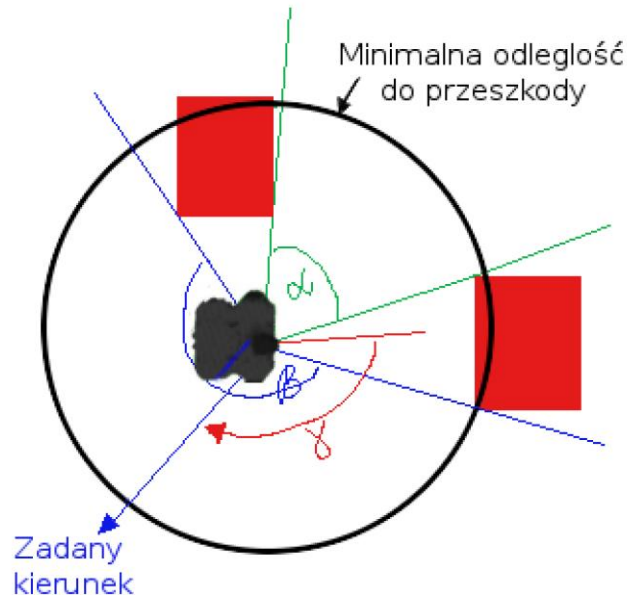
Należy uruchomić symulację robota z gotowym światem.

```
roslaunch roboty_mobilne simulation.launch
```

Do wykonania zadania należy wykorzystać dostarczonego noda (plik **angle_publisher.ipynb**) odpowiedzialnego za publikowanie różnicy pomiędzy bieżącym, a docelowym kątem. Podawane wartości są z zakresu +/- 180 stopni. W zależności od zwracanej wartości robot powinien wykonywać następujący obrót:

- dla wartości dodatnich przeciwnie do ruchu wskazówek zegara
- dla wartości ujemnych zgodnie z ruchem wskazówek zegara

Algorytm wyznaczający kąt obrotu robota dzieli pole widoku całego skanera na mniejsze obszary i wyznacza odpowiednie kąty zawierające wolną przestrzeń pomiędzy przeszkodami. Minimalna odległość od przeszkody to 30cm, poniżej tej wartości dane ze skanera oznaczają, że w tym miejscu jest przeszkoda i nie wolno tam jechać. Dla obszaru z największą szerokością określana jest dwusieczna kąta i podawany jest odpowiedni kierunek. Zwracany kąt to kąt γ odpowiadający kątowi o jaki należy obrócić robota, aby kierował się w wolną przestrzeń pozbawioną przeszkód.



Wiadomość publikowana jest na topicu `/turn_angle`.

Zadanie 3

Należy napisać w jednym skrypcie odpowiedniego Publisher'a i Subscriber'a tak, aby wykorzystać odebraną wiadomość o różnicy kąta do takiego wysterowania robota, aby cały czas poruszał się prosto, a jeżeli to konieczne wykonał odpowiedni obrót by nie wjechać w przeszkodę.

W najprostszej wersji można założyć, że jeżeli wartość bezwzględna tego kąta przekracza jakąś wartość to wtedy robot ma wykonać odpowiedni obrót w miejscu (prawo lub lewo zależnie od znaku kąta). Natomiast, jeżeli wartość bezwzględna jest mniejsza niż podany kąt to robot powinien poruszać się prosto.

Za pomocą skryptu do rejestrowania danych można zarejestrować chaotyczny przejazd, a następnie wyświetlić go na wykresie.

UWAGA!

Weryfikacja w rzeczywistości nie dotyczy maszyny wirtualnej.

Jeżeli algorytmy działają poprawnie i robot w symulacji nie wjeżdża w przeszkody można uruchomić rzeczywistego robota w przygotowanym środowisku, a następnie uruchomić niezbędne skrypty do chaotycznego omijania przeszkód.